SYSTEM ANALYSIS DOCUMENTATION (SAD)

DaddiesTrip — AI-Powered Cross-Border Travel Orchestration Platform

UMHackathon 2026 | Team SugarDaddies

github.com/SkyLee310/UMHackathon-DaddiesTrip

---

1. Introduction

This section introduces DaddiesTrip, the AI-powered cross-border travel orchestration platform developed by team SugarDaddies for UMHackathon 2026. DaddiesTrip delivers a single conversational interface that eliminates the fragmentation of group travel planning, orchestrating the complete lifecycle of a trip — from a natural language prompt to a fully structured itinerary — within seconds.

Purpose: This System Analysis Document details the technical scope, architectural decisions, and design rationale behind the development of DaddiesTrip. It serves as the definitive technical reference for all developers and evaluators reviewing the system.

The key elements of the system covered in this document are as follows:

Architecture: DaddiesTrip is a split-deployment, client-server platform. The React SPA frontend is hosted on Vercel's CDN edge network, while the FastAPI backend runs as a persistent Docker container on Railway. A four-agent LLM pipeline (Analyzer → Planner → Booking → Edge) powers all AI reasoning using the Ilmu AI API (OpenAI-compatible).

Data Flows: User prompts flow from the browser through an SSE connection to the Railway backend. The Orchestrator coordinates the four agents sequentially, emitting typed SSE events back to the frontend as each stage completes. Live FX rates are fetched from the fawazahmed0 currency CDN, and itinerary data is rendered progressively.

Model Process: The pipeline follows a strict sequential agent model: the Analyzer validates and extracts structured metadata from the user prompt; the Planner generates a day-by-day itinerary; the Booking Agent enriches it with flights, hotels, and deep-links; the Edge Agent applies deterministic Python heuristics to detect and correct known LLM hallucinations before the final result is emitted.

Role of Reference: This document is the primary reference for developers, testers, and hackathon evaluators to understand the overall system architecture, agent pipeline design, data structures, and deployment topology of DaddiesTrip.

2. Background

Group travel planning is inherently fragmented. A typical group of friends or colleagues must navigate multiple itinerary tools (Google Docs, Notion), flight comparison engines (Skyscanner, Google Flights), accommodation platforms (Booking.com, Agoda), manual multi-currency expense calculations, and messaging apps to coordinate preferences — all simultaneously.

Previous Version: No prior version of DaddiesTrip exists. This platform was conceived and built entirely within the UMHackathon 2026 hackathon period as a greenfield project.

Changes in Major Architectural Components: N/A — first release. Key architectural choices include the deliberate use of SSE over WebSockets (simpler, unidirectional, HTTP-native), Railway over Vercel serverless for the backend (no hard timeout limit for 30–90 second LLM pipelines), and a four-agent design over a single monolithic LLM call (minimises hallucinations, enables progressive rendering).

3. Target Stakeholder

Stakeholders	Roles	Expectations
Traveller / End User	Enters a natural language travel prompt; reviews, amends, and downloads the generated itinerary	One-click trip planning with flight options, cost splits, and maps — no tab-switching required
Group Members	Participate in shared cost-splitting and view the itinerary generated for the group	Transparent per-person expense breakdown with live currency conversion
Hackathon Evaluators	Assess technical feasibility, AI integration, and overall product quality	Clear architecture, live demo, working SSE stream, and accurate multi-agent pipeline
Development Team (SugarDaddies)	Design, build, and deploy the full-stack platform	Modular agent codebase, clean API contracts, reproducible Docker environment
QA / Testers	Validate agent pipeline correctness, edge-case handling, and UI responsiveness	Defined test suite (TC-01, TC-02, AI-01), clear SSE event schema, documented hallucination corrections

4. System Architecture & Design

4.1. High Level Architecture Overview

Type	Details
System	Web Application (React SPA — Desktop, Tablet, Mobile)
Architecture Pattern	Client → CDN (Vercel) → Persistent Container (Railway) → Multi-Agent LLM Pipeline → SSE Stream → Client
Architecture Style	Split-deployment, client-server with a synchronous multi-agent LLM orchestration pipeline
Backend Framework	FastAPI (async Python) — RESTful + Server-Sent Events
Frontend Framework	React 18 + Vite 5 — component-based SPA with real-time SSE client
Deployment	Frontend: Vercel CDN; Backend: Railway persistent Docker container

DaddiesTrip adopts a split-deployment architecture. The React SPA is hosted on Vercel's CDN edge network while the FastAPI backend runs as a persistent Docker container on Railway. This separation was a deliberate design decision — LLM agent pipelines routinely take 30–90 seconds, which exceeds Vercel's serverless function timeout limits. All communication between frontend and backend uses HTTP/SSE, enabling progressive rendering of results as each agent stage completes.

4.2. LLM as Service Layer

The Ilmu AI API (OpenAI-compatible, model: ilmu-glm-5.1) is integrated as the LLM service layer. The following describes its integration within the system:

Dependency Diagram:

- Prompt Construction: Each agent builds a structured system prompt + user content pair. The Analyzer prompt enforces extraction of four fields (destination, dates, participants, budget). The Planner prompt injects the extracted metadata. The Booking Agent prompt injects the draft itinerary skeleton. All prompts are hard-capped at 1,500 words before being sent to the API.
- Context Window: Each agent receives only the minimal context it needs — no agent sees the full conversation history. The Orchestrator coordinates data passing between agents via Python objects, not through the LLM context.
- Response Parsing: All LLM responses are expected in JSON. The BaseAgent class includes a JSON repair utility and a retry loop with exponential back-off (up to 3 retries) to handle malformed LLM output.
- Token Limitation: Input prompts are truncated to 1,500 words maximum before injection into the LLM. This prevents context overflow and controls token cost per session.
- API Calls between components: React Frontend calls POST /api/plan-trip-stream on Railway backend via SSE. OrchestratorAgent calls Ilmu AI API REST endpoint for each of the four agents sequentially. Ledger service calls fawazahmed0 CDN for live FX rates. Frontend calls Google Maps Embed API via iframe for activity location visualisation.
- Frontend ↔ Backend: React SPA opens an SSE connection to Railway backend. Backend streams typed events (progress, partial_itinerary, partial_flights, complete, error) as each agent completes.
- Backend ↔ Database: No persistent database. All trip state is held in-memory per request session (stateless design, acceptable for hackathon scope).
- Backend ↔ External APIs: Ilmu AI API for LLM inference; fawazahmed0 CDN for currency rates; Skyscanner and Google Flights via deep-link URL templates (no server-side API call required).

Sequence Diagram — Plan Trip Flow:

- 1. User enters prompt (e.g., '5 days in Tokyo for 4 people under RM5000') in the React SPA.
- 2. Frontend opens SSE connection to POST /api/plan-trip-stream with the prompt payload.
- 3. Backend: OrchestratorAgent invokes Analyzer Agent → validates 4 required fields; emits progress SSE event.
- 4. Backend: Planner Agent generates day-by-day itinerary skeleton; emits partial_itinerary SSE event.
- 5. Backend: Booking Agent enriches itinerary with flights, hotels, and deep-links; emits partial_flights SSE event.
- 6. Backend: Edge Agent applies hallucination correction heuristics (RM25 bug, same-airport routes, missing fields); emits complete SSE event with final FullTripObject.

- 7. Frontend progressively renders itinerary cards, flight options, cost summary, and maps as each SSE event arrives.

4.3. Technological Stack

Layer	Technology	Version	Role
Frontend	React	18.x	Component-based SPA rendering
Frontend	Vite	5.x	Build tool with fast HMR
Frontend	Tailwind CSS	3.x	Utility-first responsive styling
Frontend	Lucide React	Latest	Icon library
Backend	FastAPI	Latest	Async Python web framework + SSE
Backend	Python	3.10+	Agent logic, orchestration, data processing
Backend	Pydantic	v2	Schema validation for API I/O
Backend	Pandas	Latest	Expense aggregation and budget computation
AI / LLM	Ilmu AI API (ilmu-glm-5.1)	Latest	LLM inference — OpenAI-compatible endpoint
Infrastructure	Railway	—	Backend persistent Docker container host
Infrastructure	Vercel	—	Frontend CDN with global edge delivery
Infrastructure	Docker	Multi-stage	Reproducible build environment
Currency	fawazahmed0 Currency API	CDN	Live FX rates for 170+ currencies + offline fallback
Maps	Google Maps Embed API	Embed	Activity location iframes (no server-side key)
Voice	Web Speech API	Browser	Hands-free voice input with real-time transcription
Language	JavaScript (ES2022)	—	55% of codebase — frontend
Language	Python 3.10+	—	24% of codebase — backend agents
Language	CSS	3	15% of codebase — styling

5. Key Data Flows

5.1. Data Flow Diagram (DFD)

External Entities: End User (browser), Ilmu AI API (LLM provider), fawazahmed0 CDN (currency rates), Google Maps Embed API.

Core Processes: (1) Prompt Validation & Extraction — Analyzer Agent; (2) Itinerary Generation — Planner Agent; (3) Flight & Hotel Enrichment — Booking Agent; (4) Hallucination Correction — Edge Agent; (5) Currency Conversion & Budget Aggregation — Ledger Service.

Data Stores: In-memory Python objects per request (no persistent database). Static offline FX rate table as fallback.

Data Movement: User prompt → POST /api/plan-trip-stream → OrchestratorAgent → sequential agent pipeline → SSE stream back to React frontend. Each agent passes its structured output (Python dict/Pydantic model) directly to the next agent via the Orchestrator. No data persists between sessions (stateless).

Compliance Note: No personal data is stored. Card numbers entered in the payment simulation are validated client-side only and never transmitted to the backend or stored.

5.2. Internal Data Structures (Logical Schema)

Entity	Key Fields
FullTripObject (root)	days[], flights[], hotel{}, budget{}, currency_rates{}, split_per_person
Day	day_number, title, activities[], food[], transport, weather_note
Activity	name, location, cost_myr, duration_min, maps_query
Flight	airline, flight_number, departure, arrival, cost_myr, skyscanner_url
Hotel	name, stars, cost_per_night_myr, check_in, check_out, total_cost_myr
BudgetSummary	total_cost_myr, budget_myr, surplus_or_deficit, saving_tips[]

Note: All monetary values are stored in MYR. Currency conversion to other denominations is computed on-the-fly by the Ledger Service using live FX rates. The schema is designed for single-session use; no normalisation to 3NF is required as there is no persistent relational database.

6. Functional Requirements & Scope

This section highlights the core boundaries of the DaddiesTrip system and defines the MVP features to be demonstrated at UMHackathon 2026.

Minimum Viable Product: The following 5 core features constitute the MVP that will be built and demoed by the end of UMHackathon 2026.

#	Feature	Description
1	Conversational Trip Planning	Accept free-text (or voice) prompts describing destination, dates, group size, and budget. Validate all 4 fields before proceeding.
2	AI Itinerary Generation	Planner Agent generates a day-by-day schedule with activities, food, transport vectors, weather advice, and packing tips streamed progressively via SSE.
3	Flight & Hotel Discovery	Booking Agent enriches the itinerary with real airline options (times, pricing, Skyscanner/Google Flights deep-links) and named hotels with star ratings and MYR costs.
4	Multi-Currency Expense Splitting	Automatically divide total trip costs per person using live FX rates for 17+ currencies, with a static offline fallback.
5	Itinerary Amendment & PDF Export	Users can amend individual hotels, restaurants, or activities via natural language. The final itinerary can be downloaded as a PDF.

Non-Functional Requirements (NFRs):

Quality	Requirement	Implementation
Performance	First partial_itinerary SSE event must arrive within 15 seconds of prompt submission	Analyzer + Planner agents are optimised for speed; SSE begins streaming immediately on pipeline start
Performance	Complete pipeline (all 4 agents) must finish within 90 seconds for standard prompts	Sequential agent design with targeted prompts minimises total LLM inference time
Reliability	Edge Agent must catch and correct the RM25 repetition bug and same-airport route errors 100% of the time	Deterministic Python heuristics in EdgeAgent — no LLM call required for correction
Availability	Railway backend must maintain 99%+ uptime during hackathon evaluation window	Persistent Docker container on Railway with health check endpoint GET /api/health
Scalability	Frontend CDN must handle concurrent user sessions without degradation	Vercel global edge CDN with automatic cache invalidation and HTTPS
Security	API keys must never be committed to the repository or exposed to the client	.env.example template; keys loaded via Railway environment variables; no client-side key exposure
Security	Card input fields must use client-side validation only — no real card data stored	Card number validated via Luhn algorithm client-side; POST /api/settle only receives a card_number field for simulation

Quality	Requirement	Implementation
Usability	UI must be fully functional on mobile (iOS Safari, Android Chrome) without horizontal scrolling	Tailwind CSS responsive utility classes; tested on mobile viewports
Token Latency	LLM API calls must return within a reasonable time under normal load	5-second timeout for async LLM calls; user-facing loading state shown via SSE progress events while awaiting response
Cost Efficiency	Token usage per session must remain within a defined budget	Prompts are fixed-length templates; user input is truncated to 1,500 words before injection; no repeated queries to the LLM
Maintainability	Each agent must be a self-contained module with single-responsibility interface	Independent Python modules per agent; OrchestratorAgent is the sole coordinator
Portability	Docker build must produce an identical environment across Railway and local development	Multi-stage Docker build defined in Dockerfile; dependency isolation via requirements.txt

Out of Scope / Future Enhancements:

- Real payment processing (Stripe, FPX, etc.) — payment is simulated only
- Live flight inventory API (IATA/GDS) — LLM-enriched with Skyscanner/Google Flights deep-links
- User authentication and persistent trip history across sessions
- Native mobile applications (iOS / Android)
- Real-time group collaboration and in-app chat between group members
- Promotional offers, discount codes, or vouchers
- Scheduled delivery / future-dated trip booking

7. Monitor, Evaluation, Assumptions & Dependencies

7.1. Technical Evaluation

Grayscale Rollout & A/B Testing: For the hackathon context, the full pipeline is deployed to 100% of users. Post-hackathon, a grayscale rollout strategy would release new agent prompt changes to 5% of users initially, monitored for hallucination rates and latency, then gradually increased to 100% if no regressions are detected. A/B testing would compare different Planner Agent prompt variants to determine which produces higher-quality itineraries as rated by users.

Emergency Rollback (ER) & Golden Release: The Docker image pushed to Railway on the last stable release is tagged as the Golden Release. If the Edge Agent hallucination correction rate drops below 99% or pipeline latency exceeds 120 seconds, an emergency rollback to the Golden image is triggered manually via the Railway dashboard. Automated rollback via Railway's deployment hooks is planned as a future enhancement.

Priority Matrix:

Priority	Trigger Condition	Action
P1 — Critical	LLM pipeline failure rate > 5% or complete SSE event not emitted within 120s	Emergency rollback to Golden Release; alert team via Slack
P2 — High	Edge Agent hallucination correction rate < 99% (RM25 bug or same-airport route not corrected)	Hotfix deployed to EdgeAgent; no rollback required
P3 — Medium	Currency API CDN unavailable; live FX rates not retrievable	Automatic fallback to static offline FX rate table; no user-facing error
P4 — Low	PDF export fails for specific itinerary shapes	Log error; user advised to retry; no rollback

Monitoring: The GET /api/health endpoint returns 200 OK when the Railway backend is ready and all agent modules are loaded. For the hackathon window, the team monitors the Railway deployment dashboard for container health, memory usage, and request logs. Future monitoring would integrate Prometheus metrics and Grafana dashboards for pipeline latency per agent, LLM token usage per session, and SSE connection success rates.

7.2. Assumptions

- Users have stable internet connections on their devices during trip planning sessions.
- The Ilmu AI API (ilmu-glm-5.1) remains available and responsive within the hackathon evaluation window.
- The fawazahmed0 currency CDN is available; if unavailable, the static offline fallback covers 17 major currencies.
- Users entering prompts will provide sufficient context (destination, dates, group size, budget) for the Analyzer Agent to proceed; otherwise, a clarification SSE event is returned.
- No real financial transactions are expected — the payment flow is explicitly simulated for demonstration purposes.
- All trip planning data is ephemeral; users understand that sessions are stateless and trip history is not persisted.

7.3. External Dependencies

Tool / Service	Purpose	Risk
Ilmu AI API (ilmu-glm-5.1)	Core LLM reasoning engine — handles prompt inference, multi-step reasoning, and structured JSON output for all four agents	HIGH — if Ilmu AI API rate-limits or goes down, the core pipeline fails. Mitigation: retry logic with exponential back-off (3 retries); graceful degradation state shown to user

Tool / Service	Purpose	Risk
fawazahmed0 Currency API	Live FX rates for 170+ currencies fetched from CDN-hosted JSON	MEDIUM — if CDN is unavailable, the static offline fallback for 17 currencies is activated automatically; no user-facing failure
Google Maps Embed API	Activity location visualisation via iframe embeds — no server-side API key required	LOW — if Maps embeds fail, activity cards still render with location text; no pipeline impact
Vercel CDN	Frontend hosting with global edge delivery, automatic HTTPS, and instant cache invalidation	LOW — Vercel maintains 99.99% uptime SLA; CDN edge redundancy minimises regional failures
Railway Container Host	Persistent Docker container for FastAPI backend — no hard timeout limits	MEDIUM — container restarts may cause SSE connection drops; mitigated by Railway's automatic restart policy and the /api/health endpoint
Skyscanner / Google Flights	Deep-link URL templates for flight booking — no server-side API call	LOW — deep-links are static URL templates; no API dependency; users directed to external booking platforms

8. Project Management & Team Contributions

The duration of the preliminary round is approximately 10 days, structured as a single two-week sprint. The following timeline and team role assignments reflect the actual development plan for DaddiesTrip.

Project Timeline: (April 17 – 26, 2026)

Date	Phase	Activities
April 17	The Setup	Initialized the environment and resolved terminal pathing issues. Set up the GitHub repository and verified Python 3.10+ and Node.js 18+ compatibility.
April 18	System Design	Pivot to FastAPI for async support. Finalized the 4-Agent Pipeline architecture and defined Pydantic schemas for strict data validation.
April 19	Agent Foundations	Developed the Analyzer Agent (Prompt Extraction) and Planner Agent (Itinerary Logic) using the ilmu-glm-5.1 model.
April 20	Backend Bridge	Built the FastAPI Orchestrator and successfully implemented Server-Sent Events (SSE) for real-time streaming of partial itineraries.
April 21	Frontend Core	Built the React + Vite SPA. Integrated Tailwind CSS for the responsive SugarDaddies branded UI and conversational input.
April 22	Data Enrichment	Integrated the Booking Agent to fetch real-world metadata (Flights/Hotels) with direct booking links to Skyscanner.

Date	Phase	Activities
April 23	Ledger Logic	Developed the Multi-Currency Ledger service using @fawazahmed0/currency-api for automated expense splitting across 17+ currencies.
April 24	Safety & UX	Built the Edge Agent (Deterministic QA) to fix LLM hallucinations. Integrated Google Maps and the Web Speech API for voice input.
April 25	Deployment	Dockerized the backend for Railway to handle 90s agent timeouts. Deployed the frontend to Vercel and finalized the PyTest suite.
April 26	Submission	Final UI polish, generated printable PDF Export, and recorded the 3-minute demo video. Final submission to the UMHackathon portal.

Team Members Role:

Name	Role	Key Responsibilities
LEE MING XUAN	Team Lead & Full-Stack Architect	Overall system design, split-deployment architecture, OrchestratorAgent, SSE pipeline coordination, Railway & Vercel deployment
GOH SHENG KAI	Backend Lead — AI Agent Pipeline	Analyzer Agent, Planner Agent, prompt engineering, BaseAgent class (JSON repair, retry logic), PyTest suite
CHUAH SHANG LOONG	Backend — Booking & Edge Agents	Booking Agent (flight/hotel enrichment), Edge Agent (hallucination correction heuristics), FastAPI endpoint design
TEE HONG XUAN	Frontend Lead	React SPA, SSE client, progressive rendering, Google Maps integration, voice input (Web Speech API), responsive UI with Tailwind CSS
TAN HONG SHENG	Full-Stack — Ledger & Payment	Ledger Service (currency conversion, budget aggregation), payment simulation flow, PDF export, multi-currency splitting UI

Project Links:

GitHub Repository: https://github.com/SkyLee310/UMHackathon-DaddiesTrip

Project Timeline (Trello): https://trello.com/invite/b/69eccb57a235c02afd83d931/ATTId2ed35000a329c28fff9ae999bd919d274F7DF09/um-hackathon

Live Demo: https://daddies-trip.vercel.app/

Recommendations:

- Redis Cache: Cache the Planner Agent LLM output for identical destination + date + group size combinations for 24 hours to reduce redundant LLM calls and token costs.
- Secondary LLM Provider: Integrate a fallback LLM provider (e.g., OpenAI GPT-4o) to handle Ilmu AI API outages gracefully, using the same OpenAI-compatible interface.

- Persistent Session Storage: Introduce a lightweight Redis or SQLite store for trip history, enabling users to revisit and share itineraries across sessions.
- WebSocket Upgrade: For a future real-time group collaboration feature, upgrade the SSE stream to a bidirectional WebSocket to support multi-user simultaneous editing of the itinerary.
- Automated Monitoring: Integrate Prometheus + Grafana for pipeline latency per agent, LLM token usage per session, and SSE connection success rates in production.